Congratulations! You passed!
TO PASS 80% or higher

Keep Learning

GRADE
100%

Week 1 Quiz

LATEST SUBMISSION GRADE
100%

1. One of these statements below is true and the other three are false. Which one is true? 1 / 1 point

- ○ A Boolean variable can only be assigned a value from this set of three reserved words: {true, false, undefined}.
- ○ Every variable in C++ holds either an integer, a character, a Boolean or a floating point value (of some precision).
- ○ Every function in C++ must return a value.
- ◉ Every variable in C++ has to be associated with a specific type

✓ **Correct**
C++ is "strongly typed" which means that the type of every variable is assigned when the variable is declared, and the type of a variable cannot change once the variable is declared.

2. According to the C++ standard, what is the name of the function is the starting point for a program? 1 / 1 point

- ○ begin()
- ○ init()
- ○ start()
- ◉ main()

✓ **Correct**
When you write a C++ program, the program begins when the operating system calls the function "main()."

3. One of these statements below is true and the other three are false. Which one is true? 1 / 1 point

- ○ A class can consist of multiple member data variables, but all must be of the same type.
- ◉ A class can consist of multiple member data variables of different types, but each type must be specified when the class is defined.
- ○ A class can consist of multiple member data variables of different types, but each member variable must be one of the built-in types.
- ○ A class can consist of multiple member data variables, but the type of each data variable does not need to be specified until the class is used to declare a variable.

✓ **Correct**

4. One of these statements below is true and the other three are false. Which one is true? 1 / 1 point

- ○ The member functions of a class can only operate on member data variables of that class.
- ○ The member data variables in a class can only be accessed by the member functions of that class.
- ○ Any functions that operate on a class's member data variables must be implemented independent of the class in a separate .cpp file.
- ◉ The member functions of a class always have access to every member data variable of that class.

✓ **Correct**

5. Which C++ directive is used to insert the contents of another file at the current location while processing the current file? 1 / 1 point

- ○ #library
- ◉ #include
- ○ #import
- ○ #using

✓ **Correct**

The file indicated by the string after the "#include" directive is compiled before the rest of the current file is compiled.

6. **Given only the following code:** 1 / 1 point

```
namespace uiuc {
  class Pair {
    double a,b;
  };
}
```

which of the following syntax can be written outside of the namespace declaration to properly create a variable named "p" of type Pair?

- (•) uiuc::Pair p;
- ( ) Pair p;
- ( ) uiuc/Pair p;
- ( ) (uiuc) Pair p;

✓ **Correct**

We have to reference the namespace to access the classes defined in it.

7. **Which keyword is used to indicate which namespace(s) to search to find classes and variables when they are referenced throughout the rest of the program?** 1 / 1 point

- (•) using
- ( ) std
- ( ) namespace
- ( ) uiuc

✓ **Correct**

The "using" keyword indicates to the compiler from which namespace references to classes and variables should be found.

8. **Why do we use namespaces in C++ programming?** 1 / 1 point

- ( ) Because all references to variable and class names must be made through namespace.
- (•) Because two different libraries might use the same label for a class or variable
- ( ) Because it allows a library to claim a variable or class name that cannot be used by any other library.
- ( ) Because all variable and class names must be defined using a namespace.

✓ **Correct**

Namespaces allow different libraries to use the same label for a class or variable because they can each define a unique namespace to differentiate them when they are used together in a program.

9. **What is the namespace of the C++ Standard Library?** 1 / 1 point

- ( ) csl
- ( ) stdio
- (•) std
- ( ) cstl

✓ **Correct**

10. **Which operator is used to send a sequence of strings, numbers and other values to the standard library's cout object in a specific order so that they will be printed to the console?** 1 / 1 point

- ( ) =
- (•) <<
- ( ) &
- ( ) +

✓ **Correct**

This is called the "streaming" operator and sends the operand on its right to the stream on its

left. Syntactically, the expression also evaluates to the stream on the left, so the line:

```
std::cout << "a" << 3;
```

is first evaluated as

```
(std::cout << "a") << 3;
```

and the expression in the parentheses returns a reference to cout after sending “a” to it, so that the second “<<” operator sends the value 3 to cout. You can think of it like this:

```
// After the "a" has been sent to std::cout,
// the rest of the expression evaluates as:
std::cout << 3;

// Ater the 3 has been sent:
std::cout;
```

After each "<<" is evaluated, the sub-expression evaluates to a reference to std::cout, including at the very end, after all the streamed items have been sent. This will be useful to know later, when we'll show you how to make your own classes compatible with streaming to std::ostream objects (like std::cout).

Congratulations! You passed!
TO PASS 80% or higher

Keep Learning

GRADE
100%

Week 2 Quiz

LATEST SUBMISSION GRADE
100%

1. **Recall that every variable in C++ has these four things: a name, a type, a value and a memory location.** — 1 / 1 point

```
1  int *p;
2  p = new int;
3  *p = 0;
4
5
```

For the code above, which one of the following is NOT true for variable p?

- ○ The name of the variable is "p"
- ○ The type of the variable is a pointer to an integer, specifically the type "int *"
- ◉ The value of the variable is 0
- ○ The memory address of the variable is the value returned by the expression &p

✓ **Correct**
Even though p points to a memory address containing the integer value 0, the value of the pointer is the memory address itself.

2. **Which one of the following is true?** — 1 / 1 point

- ○ The C++ statement "int i;" allocates memory for one integer on the heap.
- ○ The "new" operator allocates memory on the stack that gets removed from the stack by the "delete"operator.
- ◉ The address of any memory location in the stack is larger than the address of any memory location in the heap.
- ○ You should avoid using the memory address 0x0 for pointers whose value is not yet set, because memory location 0x0 is a valid location for the system to allocate to hold the contents of a variable.

✓ **Correct**
The stack begins at a high memory address and works its way down, whereas the heap begins at a low memory address and works its way up.

3. **Suppose we are writing the following function that is intended to return a pointer to a location in memory holding an integer value initialized to zero.** — 1 / 1 point

```
1  int *allocate_an_integer() {
2      // declare variable i here
3      *i = 0;
4      return i;
5  }
6
7
```

How should variable i be declared?

- ○ int *i;
- ◉ int *i = new int;
- ○ int j;

  int *i = &j;
- ○ int i;

✓ **Correct**
The variable i should be a pointer to a memory location to an integer allocated from the heap so the memory location continues to be allocated after the function has returned.

4. **Suppose we have this alternative function that returns a pointer to a memory location to an integer value of zero.** — 1 / 1 point

```
1  int *allocate_an_integer() {
2      int i = 0;
3      return &i;
```

```
}

int main() {
    int *j;
    j = allocate_an_integer();
    int k = *j;
    return 0;
}
```

What value is variable k assigned and why?

- Assuming that the program compiles with just a warning and not an error due to the settings, the variable k will not be assigned a value, because the running program will crash the whole operating system.
- Variable k is certainly assigned the value zero, because the C++ runtime will automatically move the local variable to the heap and return the address of that heap variable instead.
- Unknown. Depending on the compiler settings, the compiler may report that a local variable address is being returned, which could be treated as a warning or as a compilation error; Or, if the program is allowed to compile, then at runtime the variable k could be assigned zero, or some other value, or the program may terminate due to a memory fault.
- Variable k is not assigned a value, because even if the compiler is set to ignore warnings and continue with compilation, the compiled program will still automatically detect that a local variable's address is being used after the function has returned, and exit to the operating system with a non-zero error code.

Correct

Correct. Variable i is allocated with a memory location on the stack. When allocate_an_integer() returns, its memory on the stack including the memory holding the value of i is freed and may be used for other purposes, and may be overwritten with a new value.

5. Suppose we declare a variable as "int i;" Which of the following expressions returns the address of the memory location containing the contents of variable i? 1 / 1 point

- i.addr
- *i
- &i
- i->addr

Correct

The & operator returns the address of its operand.

6. 1 / 1 point

```
int i = 0;
int *j = &i;
```

How many memory allocations are made on the stack and on the heap for the above code? For example, declaring an integer would count as one memory allocation.

- Zero allocations on the stack and one allocation on the heap.
- Zero allocations on the stack and two allocations on the heap.
- One allocation on the stack and one allocation on the heap.
- Two allocations on the stack and zero allocations on the heap.
- One allocation on the stack and zero allocations on the heap.

Correct

Two allocations are made, with both to the stack: one for the integer i and one for the memory address that is the value of pointer j.

7. 1 / 1 point

```
int *i = new int;
```

How many memory allocations are made on the stack and on the heap for the above code? For example, allocating space for one integer would count as one memory allocation.

- Two allocations on the stack and zero allocations on the heap.
- Zero allocations on the stack and two allocations on the heap.
- One allocation on the stack and one allocation on the heap.
- Zero allocations on the stack and one allocation on the heap.
- One allocation on the stack and zero allocations on the heap.

✓ **Correct**

The use of the new operator allocates memory on the heap that persists until it is deallocated by the delete operator, instead of on the stack which is deallocated when the current function returns.

8. 1 / 1 point

```
1  int *i = new int;
2  *i = 0;
3  int &j = *i;
4  j++;
5
6
```

What does the last line of the above code segment do?

- ○ Increments the value of j by one, where the value of j is a local copy stored on the stack of the value of i stored on the heap.
- ◉ Increments the value pointed to by variable i by one.
- ○ Increments the address pointed to by variable i by one.
- ○ Causes an error.

✓ **Correct**

Yes, j is a direct reference to the same actual integer that i points to indirectly.

9. 1 / 1 point

```
1  int i = 0, j = 1;
2  int *ptr = &i;
3
4  i = 2;
5  *ptr = 3;
6  ptr = &j;
7  j = i;
8  *ptr = 4;
9
10
```

Enter the number of different values stored in the same address that variable i has during the execution of the code above. (Your answer should be a single integer, which is the total number of different values assigned to that address.)

3

✓ **Correct**

The value of i is set to zero at line 1, then two at line 4, then three at line 5 because of line 2.

10. 1 / 1 point

```
1  class Pair {
2      public: double a,b;
3  };
4
5  int main() {
6      Pair *p = new Pair;
7      p->a = 0.0;
8      return 0;
9  }
10
11
```

The expression p->a is equivalent to which one of the following?

- ◉ (*p).a
- ○ p.a
- ○ *(p.a)
- ○ p.*a

✓ **Correct**

The arrow operator -> accesses the right operand member in the class at the memory address of its left operand.

Congratulations! You passed!
TO PASS 80% or higher

Keep Learning

GRADE
100%

Week 3 Quiz

LATEST SUBMISSION GRADE
100%

1. Which one of these is the only true statement about a class constructor. 1 / 1 point

- A class can only have one constructor.
- When declaring a constructor for a class, the return type of the constructor must be the type of the class.
- (selected) When declaring a constructor for a class, the name of the constructor must be the name of the class.
- A class must have at least one constructor declared for it.

Correct
A constructor cannot be assigned a name other than that of the class itself.

2. Which of the following examples does NOT call a copy constructor at least once? 1 / 1 point

(If you already have advanced knowledge of C++ that makes this seem like a trick question, then we'll also specify this: Assume that compiler optimizations are mostly disabled.)

(selected)
```
1  Cube a,b(10);
2  a = b;
```

```
1  // Function prototype for "intersect":
2  Cube intersect(Cube &left, Cube &right);
3  // ...
4  Cube a(10),b(5);
5  Cube c;
6  c = intersect(a,b);
7
```

```
1  Cube b(10);
2  Cube a = b;
```

```
1  // Function prototype for "contains":
2  int contains(Cube outer, Cube inner);
3  // ...
4  Cube a(10),b(5);
5  int a_bounds_b = contains(a,b);
6
```

Correct
Correct: The copy constructor will not be called.

In this case, the assignment operator is called (either the default assignment operator or one that has been explicitly declared). Since the operand of the assignment operator is passed by reference and not by value, the copy constructor is not called because no new object needs to be constructed.

3. Recall that a custom assignment operator can be declared such that line 2 of the code below executes a user-defined function to perform the assignment. 1 / 1 point

```
1  Cube a,b(10);
2  a = b(10);
```

Which one of the following statements regarding the declaration of such a custom assignment operator allowing is true?

- (selected) The custom assignment operator is a public member function of the class.
- The custom assignment operator function is declared with two arguments: the source and target objects of the assignment.
- The type of the custom assignment operator function should be void.
- The custom assignment operator is a function declared with the name "operator::assignment".

Correct
Indeed, the assignment operator is a member function of the class of the target object, and this member function must be public in order for the assignment operator to be used in contexts outside of the class's own implementation as shown in the example in this question.

4. **Consider the following class:** 1 / 1 point

```
class Orange {
  public:
    Orange(double weight);
    ~Orange();
    double getWeight();

  private:
    double weight_;
};
```

Select all functions that are present in this class (including any automatic/implicit functions added by the compiler):

- [ ] Default constructor
- [x] At least one custom, non-default constructor

✓ Correct

- [x] Copy constructor

✓ Correct

- [x] Assignment operator

✓ Correct

- [x] Destructor

✓ Correct

5. **Consider the following class:** 1 / 1 point

```
class Blue {
  public:
    double getValue();
    void setValue(double value);

  private:
    double value_;
};
```

Select all functions that are present in this class (including any automatic/implicit functions added by the compiler):

- [x] Default constructor

✓ Correct

- [ ] At least one custom, non-default constructor
- [x] Copy constructor

✓ Correct

- [x] Assignment operator

✓ Correct

- [x] Destructor

✓ Correct

6. **Consider the following class:** 1 / 1 point

```
class Animal {
  public:
    Animal();
    Animal(std::string name);
    Animal(std::string name, int age);
    Animal(std::string name, int age, double weight);

    Animal(const Animal & other);

    void setName(std::string name);
    std::string getName();

  private:
```

```
14    // ...
15 };
16
```

How many explicit (non-automatic) constructors are present in the class?

- ○ 2
- ○ 3
- ○ 4
- ◉ 5
- ○ 6
- ○ 7

✓ **Correct**

7. **When you use the new operator to create a class object instance in heap memory, the new operator makes sure that memory is allocated in the heap for the object, and then it initializes the object instance by automatically calling the class constructor.** **1 / 1 point**

After a class object instance has been created in heap memory with new, when is the *destructor* usually called?

- ○ The destructor is called automatically when the program returns from the function where the **new** operator was used to create the class object instance.
- ◉ The destructor is called automatically when the **delete** operator is used with a pointer to the instance of the class.
- ○ The destructor is called automatically when the variable goes out of scope.
- ○ The programmer always needs to call the destructor manually in order to free up memory.

✓ **Correct**

8. **Consider the following program:** **1 / 1 point**

```
1  double magic(uiuc::Cube cube) {
2    cube.setLength(1);
3    return cube.getVolume();
4  }
5
6  int main() {
7    uiuc::Cube c(10);
8    magic(c);
9    return 0;
10 }
11
```

How many times is the uiuc::Cube 's copy constructor invoked?

- ○ Never
- ◉ Once
- ○ Twice
- ○ Three times

✓ **Correct**

9. **We have looked at examples where the assignment operator returned the value "*this". The variable "this" is available by default in most class member functions. What is the value of this built-in class variable "this"?** **1 / 1 point**

- ◉ A pointer to the current object instance.
- ○ An alias of the current object.
- ○ A reference to the current object.
- ○ A pointer to a heap-memory copy of the current object.

✓ **Correct**

This is correst. In fact, members of the current object can be accessed as "this->*membername*" . For example, if you define a member function whose argument is the same name as a member variable, any use of that name in the local scope of the function refers to the argument and not the member variable, but you can still access the member variable as "this->*membername*" . Hence the following example works.

```
1  class Just_a_double {
2      double val;
3  public:
4      void setValue(double val) {
5          this->val = val;
6      }
7  }
```

10. **Consider the code below that includes a class that has a custom constructor and destructor and both utilize a global variable (which has global scope and can be accessed anywhere and initialized before the function main is executed).**

1 / 1 point

```
int reference_count = 0;

class Track {
public:
    Track() { reference_count++; }
    ~Track() { reference_count--; }
};
```

Which one of the following procedures (void functions) properly ensures the deallocation of all the memory allocated for objects of type Track so the memory can be re-used for something else after the procedure returns?

For the correct answer, the variable reference_count should be zero after all calls to track_stuff() and all of the memory should be deallocated properly. This will dependably occur after only one of the following procedures.

- ```
  void track_stuff() {
      Track *t = new Track;
      // ...
      t->~Track();
      return;
  }
  ```

- ```
  void track_stuff() {
      Track t;
      // ...
      delete t;
      return;
  }
  ```

- ```
  void track_stuff() {
      Track t;
      Track *p = &t;
      // ...
      delete p;
      return;
  }
  ```

- ```
  void track_stuff() {
      Track t;
      Track *p = new Track;
      // ...
      delete p;
      return;
  }
  ```

✓ **Correct**

This procedure declares t as an object of class Track with its memory allocated on the stack at local scope. The object t will automatically be destroyed and the memory will be deallocated when the function returns. This example also allocates memory for another Track on the heap using **new** and stores that address in the pointer to Track, p. The object p is destroyed and its memory is deallocated when the **delete** operator is used.

Congratulations! You passed!

TO PASS 80% or higher

Keep Learning

GRADE
90%

Week 4 Quiz

LATEST SUBMISSION GRADE
90%

1. **Which one of the following is NOT true?** — 1 / 1 point

- (•) C++ allows a local variable to be declared in main() with an unknown type that can be defined when the program is executed.
- ( ) C++ allows a member variable to be declared in a user-defined class with an unknown type that can by defined when an object of that class is created.
- ( ) C++ allows a variable to be declared in a user-defined member function of a user-defined class that can be defined when the function is called.
- ( ) C++ allows a variable to be declared in a user-defined function with an unknown type that can be defined when the function is called.

✓ **Correct**
Even though C++ allows functions and classes to use templated types that are defined when the function is called or an object of that class is created, every variable must have a type known at compile time.

2. **Suppose you want to create a vector of integers. Which of the following creates an instance of the std::vector class that can contain integers?** — 1 / 1 point

- ( ) int<std::vector> v;
- ( ) int v[256];
- ( ) int *v;
- (•) std::vector<int> v;

✓ **Correct**
This correctly creates a std::vector using "int" as the template type of the elements of the vector.

3. **Which of the following will generate an error at compile time?** — 1 / 1 point

- ( ) std::vector<char[256]> v;
- ( ) std::vector<double> v;
- ( ) std::vector<std::vector<int>> v;
- (•) std::vector v;

✓ **Correct**
This will generate a compile-time error because the compiler does not know what type should be used for the elements of the std::vector, and every variable (including v) has to have a type at compile time. You have to supply a type as the template parameter for the elements of the std::vector.

4. — 1 / 1 point

```
template <typename Type>
Type max(Type a, Type b) {
    return (a > b) ? a : b;
}
```

Which one of the following exampled is a proper way to call the max function declared above in template form?

- ( ) <Type = double>max(5.0,10.0)
- (•) max(5.0,10.0)
- ( ) max<double>(5.0,10.0)
- ( ) max<Type = double>(5.0,10.0)

✓ **Correct**
Whereas a class needs to explicitly identify the type, a templated function does not need to explicitly identify the type(s) used if the type of its arguments can be sufficiently matched to the templated types used in the function declaration.

5. 1 / 1 point

```
template <typename Type>
Type max(Type a, Type b) {
    return (a > b) ? a : b;
}

class Just_a_double {
public:
    double num;
};

int main() {
    Just_a_double a,b;
    a.num = 5.0;
    b.num = 10.0;
    ...
}
```

Given the above code, which one of the expressions below, if used at line 15, will compile and not generate a compile error?

- max("five",10.0)
- (•) max(a.num,b.num)
- max(a,10.0)
- max(a,b)

✓ **Correct**

Both arguments to max() are the same type and both can be compared using the greater-than operator.

6. **Which one of the following properly declares the class RubikCube derived from the base class Cube?** 1 / 1 point

- class Cube : public RubikCube {...};
- class Cube(RubikCube) {...};
- class RubikCube(Cube) {...};
- (•) class RubikCube : public Cube {...};

✓ **Correct**

This correctly derives RubikCube as a specialization of base class Cube.

7. 1 / 1 point

```
class Pair {
public:
    double a,b;
    Pair(double x, double y) { a = x; b = y; }
};
```

If a class equalPair is derived from the above base class (but specializes it by adding a single boolean "isequal" member variable) then which one of the options below is a proper declaration of a constructor for equalPair?

(As a side note: Although the member variables are of type double, for the sake of this question, we are not concerned about making approximate comparisons of floating-point types, only exact comparisons. Usually, in practical usage, when you compare floating-point values, you should write a function for *approximate* comparison. That is, you should allow numbers to be considered equal if they have a very small absolute difference, even if they are not exactly the same.)

-
```
equalPair(double a, double b) {
    isequal = (a == b);
}
```

- (•)
```
equalPair(double a, double b) : Pair(a,b) {
    isequal = (a == b);
}
```

-
```
equalPair(double a, double b) {
    this->Pair(a,b);
    isequal = (a == b);
}
```

-
```
equalPair(double a, double b) {
    Pair(a,b);
    isequal = (a == b);
}
```

✓ **Correct**

Correct. The class Pair does not have a default constructor so it needs to be explicitly constructed using Pair(a,b).

8. 0 / 1 point

```
class Pair {
private:
    double a,b;
};

class equalPair : public Pair {
private:
```

```
    bool isequal;
public:
    int status();
}
```

When the function status() is implemented, which variables will it have access to?

- No member variables of either equalPair or Pair.
- Just the member variables a,b of Pair.
- Just the member variable isequal of equalPair.
- Both the member variables a,b or Pair and isequal of equalPair.

Incorrect

The derived class equalPair still has access to its own private members, but even though Pair is indicated as a public base class, the derived class equalPair does not have access to the private members of Pair.

9. 1 / 1 point

```
1  class Just_a_double {
2  public:
3      double a;
4
5      Just_a_double(double x) : a(x) { }
6      Just_a_double() : Just_a_double(0) { }
7  }
```

Which constructors, if any, compile properly?

- The constructor on line 5 results in a compiler error but the constructor on line 6 compiles properly,
- The constructor on line 5 compiles properly, but the constructor on line 6 results in a compiler error.
- Both constructors on lines 5 and 6 result in compiler errors.
- Both constructors on lines 5 and 6 compile properly

Correct

The initializer lists allow both member variable constructors as well as other declarations of the class constructor.

10. **C++ is ...** 1 / 1 point

- ... a great language for programming data structures.
- ... the greatest language for programming data structures ever!
- ... meh.

Correct

Congratulations! You passed!
TO PASS 80% or higher

Keep Learning

GRADE
97.5%

Week 1 Quiz

LATEST SUBMISSION GRADE
97.5%

1.

```
1  int tri(int n) {
2      int i,j;
3      int count = 0;
4
5      for (j=0; j < n; j++)
6          for (i=0; i < j; i++)
7              count++;
8
9      return count;
10 }
```

1 / 1 point

Perform a run-time analysis of the code above. Express the number of times the variable *count* is incremented in terms of "Big Oh" notation.

Recall that "Big Oh" notation is denoted as O() where the parameter of O() is a simple function of n that indicates how the run-time increases as n increases. For example, if the run-time grows as a polynomial of n, such as "5n^3 + 3n^2" then the "Big-Oh" notation would ignore constants and lower growing terms and simply state O(n^3) growth.

- ○ O(n^2 + n)
- ◉ O(n^2)
- ○ O((1/2) n^2)
- ○ O(1)

✓ **Correct**
Even though the number of times the variable count is incremented is n*(n-1)/2 = (1/2) n^2 - (1/2) n, the "Big Oh" notation is only concerned about the order of the growth, which is the growth of the highest degree term (e.g. n^2) ignoring any constant factors of that term (e.g. 1/2).

2. **You have an array that is currently length one and already contains one item. You need to implement a function Append(i) that adds the item i to the position after the current last item of the array. If the array is full, then your Append() function will need to expand the size of the array so that it can store the additional item i. Recall that expanding the size of an array requires allocating new memory for the expanded size and copying all of the current array items to the new (expanded) array before de-allocating the previous (full) array.**

1 / 1 point

(It is okay to assume there is always enough memory to allocate for an array.)

Your Append() function will be called an unknown number of times. Which method for resizing the array would result in the fastest total run-time for calling Append() n times to add n items to the array?

- ○ Increasing the length of the array by one every time an item is added.
- ○ Expanding the array to length n + 1 the first time Append() is called.
- ◉ Doubling the length of the array every time an item is added when the array is already full.
- ○ Increasing the length of the array by one billion every time an item is added and the array is already full.

✓ **Correct**
Recall that this strategy requires O(n) operations instead of the others which require O(n^2) to add n items to the array.

3. **Which one statement below is FALSE? Assume we are using the most efficient algorithms discussed in lecture.**

1 / 1 point

- ○ Adding n items, one at a time, to the end of an array takes O(n) time overall.
- ○ Finding an item in a sorted linked list of n items takes O(n) time.
- ◉ Finding an item in a sorted array of n items cannot be done in better than O(n) time.
- ○ Adding n items, one at a time, to the front of a linked list takes O(n) time overall.

✓ **Correct**
This is indeed the false statement, because If the array is sorted, then one can perform a binary search of the array by going to the middle of the array to see if that is the correct item. If not, then the correct item would be one one side or the other, such that only half of the array needs to be search. Hence the search space is cut in half each step and O(lg n) steps are needed.

(Recall "lg n" is the base 2 logarithm of n, such that if x = lg n, then n = 2^x. If x is not an integer, then O(lg n) would refer to the ceiling of lg n.)

4. **You have a list of 100 items that are not sorted by the item value. Which one task below would run much faster on a list implemented as an array rather than implemented as a linked list?** — 1 / 1 point

- ○ Inserting a new item between the 24th item and the 25th item.
- ◉ Replacing the 25th item in the list with a different item.
- ○ Searching the list for all items that match a given item.
- ○ Finding the first item.

✓ **Correct**

Once you have the 25th item in the list, replacing it with a different item takes just a single step for both an array and a linked list. However, the array can just compute the index (or address) of the 25th item in a single step with a simple formula, whereas a linked list would require 24 steps to follow 24 links past the first item at the head of the list.

5. **You have a list of 100 items that are not sorted by the item value. Which one task below would run much faster on a list implemented as linked list rather than implemented as an array?** — 1 / 1 point

- ○ Finding the first item
- ◉ Inserting a new item between the 24th item and the 25th item.
- ○ Replacing the 25th item in the list with a different item.
- ○ Searching the list for all items that match a given item.

✓ **Correct**

Inserting a new item in the middle of the list implemented as an array requires copying each of the remaining items (25th to 100th) to new locations (26th to 101st), and may require resizing the array. Inserting a new item in the middle of the list implemented as a linked list would require setting the "next" pointer at the 24th item to the new item, and the "next" pointer at the current item to the (previously) 25th item.

6. **Suppose you want to implement a queue ADT using a linked list. Your queue needs to be able to "push" (or "enqueue") a single item in constant time, as well as "pop" (or "dequeue") a single item in constant time. The operations need to happen at opposite ends of the queue, as would be expected of the queue ADT. However, the people who use your queue implementation don't need to know about how exactly it is implemented, so you can be somewhat creative in how you implement it, as long as the "push" and "pop" operations behave as expected. Which of the following implementations can accomplish this? Select all that apply. (For the sake of this question, let's not consider any design strategies that would close the linked list into a circle.)** — 0.75 / 1 point

- ☑ You can do it with a singly-linked list where the list has only a "head" pointer and each node has only a "next" pointer.

! **This should not be selected**

Incorrect; this implementation won't work, because with only the "head" pointer, you only have access to one end of the list in O(1) time. If you want to access the other end of the list, you would need to start at the head and traverse the whole list to the end using the "next" pointers, and this would take O(n) time just to identify the memory address of the other end of the list. Therefore you can't implement both the "push" and "pop" methods in O(1) with this design.

- ☑ You can do it with a modified singly-linked list where the list has both a "head" pointer and a "tail" pointer, but each node has only a "next" pointer.

✓ **Correct**

Yes, if you choose to "push" (or "enqueue") incoming items at the tail end of the list, then you can use the "tail" pointer and the tail node's "next" pointer to append a new item to the tail in O(1). You can use the "head" pointer and the head node's "next" pointer to "pop" (or "dequeue") the existing head item in O(1) while updating the head pointer to point to the next item remaining after it in the list. But there is also another implementation option here that could be used.

- ☐ You can't implement a queue as a linked list. You need a more advanced data structure.
- ☑ You can do it with a doubly-linked list where the list has a "head" pointer and a "tail" pointer and each node has a "next" pointer and a "previous" pointer.

✓ **Correct**

Yes, a full doubly-linked list implementation can do it, because the "head" and "tail" pointers give you O(1) access to both ends of the list, and the "next" and "previous" pointers also allow you to add or remove nodes at either end of the list easily in O(1). But there is also a less memory-intensive implementation that can be used here.

7. **When implementing a queue which will need to support a large number of calls to its push() and pop() methods, which choice of data structure results in a faster run time according to "Big Oh" O()** — 1 / 1 point

analysis?

- ○ A linked list because an array cannot be used to implement a queue that supports both push() and pop() methods.
- ○ The array implementation of a queue has a better run time complexity than does the linked list implementation.
- ◉ Both array and linked list implementations of a queue have the same run time complexity.
- ○ The linked list implementation of a queue has a better run time complexity than does the array implementation.

✓ **Correct**

The push() and pop() methods can be implemented in constant time O(1) for both an array and a linked list. Push and pop run in amortized constant time O(1) for an array, but the large number of calls to push() and pop() will ensure the cost of resizing the array is properly amortized by the benefits of the array.

8. **When implementing a stack which will need to support a large number of calls to its push() and pop() methods, which choice of data structure results in a faster run time according to "Big Oh" O() analysis?** — 1 / 1 point

- ○ The linked list implementation of a stack has a better run time complexity than does the array implementation.
- ○ The array implementation of a stack has a better run time complexity than does the linked list implementation.
- ○ An array implementation because a linked list cannot be used to implement a stack that supports both push() and pop() methods.
- ◉ Both array and linked list implementations of a stack have the same run time complexity.

✓ **Correct**

The push() and pop() methods can be implemented in constant time O(1) for both an array and a linked list. Push and pop run in amortized constant time O(1) for an array, but the large number of calls to push() and pop() will ensure the cost of resizing the array is properly amortized by the benefits of the array.

9. **Suppose this stack is implemented as a linked list.** — 1 / 1 point

```
1  std::stack<int> s;
2  s.push(1);
3  s.push(2);
4  s.push(3);
5
```

What is the value at the head of the linked list used to implement the stack s?

- ○ 2
- ◉ 3
- ○ 1
- ○ A stack cannot be implemented using a linked list.

✓ **Correct**

When implementing a stack as a linked list, a call to push() method creates a new node, sets the *head* pointer of the linked list to this new node, and the new node's *next* pointer points to the previous head node of the linked list. Since this value is the last value pushed onto the stack, it indeed will be placed at the head of the list.

10. **Suppose we had the following interface for a stack and queue, along with a correct implementation.** — 1 / 1 point

(Note that in this simple version, the "pop" and "dequeue" methods will remove an item and also return a copy of that same item by value. This is a little different from how the C++ Standard Template Library implementations of a stack and queue work. In STL, you have separate functions for peeking at the next value that would be removed, and for actually removing the item.)

```
1  class Stack{
2    public:
3      Stack();
4      bool push(int x);
5      int pop();
6      bool isEmpty();
7      // other lines omitted
8  };
9
10 class Queue{
11   public:
12     Queue();
13     bool enqueue(int x);
14     int dequeue();
15     bool isEmpty();
16     // other lines omitted
17 };
18
```

What output does the following code produce?

```
1  main() {
```

```
    Stack s = Stack();
    Queue q = Queue();

    for(int i = 0; i < 5; i++){
      s.push(i);
      q.enqueue(i);
    }

    for(int i=0; i < 5; i++){
      s.push(q.dequeue());
      q.enqueue(s.pop());
    }

    while (!q.isEmpty())
        std::cout << q.dequeue() << " ";
}

```

- 0 1 2 3 4 4 3 2 1 0
- 4 3 2 1 0 0 1 2 3 4
- 4 3 2 1 0
- 0 1 2 3 4

Correct

Congratulations! You passed!
TO PASS 80% or higher

Keep Learning

GRADE
100%

Week 2 Quiz

LATEST SUBMISSION GRADE
100%

1. **For a linked structure of edges and nodes to be a tree, which of the following is not required to be true?** — 1 / 1 point

- ○ If any two nodes are connected, they are connected by only one path of unique nodes and edges.
- ○ Every node is connected to every other node by some path of edges.
- ○ Every node has a parent except for one single root node.
- ◉ Every node has zero, one or two children.

✓ **Correct**
This is true of a **binary** tree, but not of a tree in general.

2. **Which data structure below supports the fastest run time for finding an item in a sorted list of items?** — 1 / 1 point

- ◉ Array
- ○ Linked List
- ○ Binary Search Tree
- ○ All of these data structures have the same run time complexity for finding an item in a sorted list of items.

✓ **Correct**
A sorted array has a worst-case run time of O(lg n) which is faster than the other options.

3. **What is the height of the binary search tree created by inserting the following values one at a time in the following order of insertion: 1 2 3 4 5 6 7 ?** — 1 / 1 point

- ○ 2
- ○ 3
- ◉ 6
- ○ 7

✓ **Correct**
The height of a tree is the number of edges from the root to its fathest descendant. Since these values are inserted one at a time and are already in sorted order, the binary search tree ends up resembling a linked list from the root following the right child link through the list, with six edges.

4. **Which of the following is NOT true of a perfect binary search tree of a list of n ordered items?** — 1 / 1 point

- ○ If the height of the tree is h, then n = 2^(h+1) - 1.
- ○ All of the leaf nodes are at the same level.
- ○ Every non-leaf node has two children.
- ◉ The worst-case run time to find an item is O(n).

✓ **Correct**
The worst-case run time to find an item in a perfect binary search tree is O(lg n), because its height h = lg(n+1) - 1.

5. **Which of the following is NOT a full binary tree?** — 1 / 1 point

- ○ A single node.
- ◉ The binary tree consisting of the subtree of ancestors of any node in any perfect binary tree.
- ○ The binary search tree created by inserting the following values one at a time: **4 2 3 5 1**.
- ○ A perfect binary tree

A perfect binary tree.

✓ **Correct**

Every node in a full binary tree has zero or two children, whereas the non-leaf nodes of the subtree of ancestors would consist of nodes each having only a single child.

6. **Which of the following is not a true statement about a complete binary tree?** **1 / 1 point**

- No node in a complete binary tree has only a right child.
- The height of a complete binary tree of n nodes is floor(lg n).
- The worst-case run time for finding an object in a complete binary search tree of an ordered list of n items is O(lg n).
- Any tree that contains a node with a single child is not a complete binary tree.

✓ **Correct**

A complete binary tree can have one node with a single child.

7. **Which one of the following functions outputs the keys of a binary search tree in item order when the root node is passed to it as its parameter.** **1 / 1 point**

```
void print(TreeNode *node){
    if (!node) return;
    std::cout << node->key << " ";
    print(node->left);
    print(node->right);
}
```

```
void print(TreeNode *node){
    if (!node) return;
    print(node->left);
    std::cout << node->key << " ";
    print(node->right);
}
```

```
void print(TreeNode *node){
    if (!node) return;
    print(node->left);
    print(node->right);
    std::cout << node->key << " ";
}
```

- None of these outputs all of the keys of the binary search tree in item order.

✓ **Correct**

This is an in-order traversal that prints the values of the descendants of the node's left child, then the node's value, then the values of the descendants of its right child.

8. **Consider the binary search tree built by inserting the following sequence of integers, one at a time: 5, 4, 7, 9, 8, 6, 2, 3, 1** **1 / 1 point**

Which method below will properly remove node 4 from the binary search tree?

- Set the *left* pointer of node **5** to **nullptr**, and then delete node **4**.
- Set the *left* pointer of node **5** to point to the node pointed to by the *left* pointer of node **4**, and then delete node **4**.
- Find the in order predecessor (IOP) of node **4**, which is node **3**. Remove node **3** from the tree by setting the right pointer of its parent (node **2**) to **nullptr**. Then copy the key and any data from node **3** to node **4**, turning node **4** into a new node **3**, and delete the old node **3**.
- Find the in order predecessor (IOP) of node **4**, which is node **3**. Remove node **3** from the tree by setting the right pointer of its parent (node **2**) to point to the node pointed to by the left pointer of node **3**. Then copy the key and any data from node **3** to node **4**, turning node **4** into a new node **3**, and delete the old node **3**.

✓ **Correct**

This is the correct way to remove node **4** from the binary search tree because node 4 has only one child, its left child.

9. **Suppose that we have numbers between 1 and 1000 in a binary search tree and we want to search for the number 363. Which of the following sequences can <u>not</u> be the sequence of nodes visited in the search?** **1 / 1 point**

- **2, 252, 401, 398, 330, 344, 397, 363**
- **925, 202, 911, 240, 912, 245, 363**
- **2, 399, 387, 219, 266, 382, 381, 278, 363**
- **924, 220, 911, 244, 898, 258, 362, 363**

✓ **Correct**

- 925 is the root

- 925 is the root.
- 202 is 925's left child.
- 911 is 202's right child.
- 240 is 911's left child.
- 912 cannot be any child of 240 because that would place it in the left subtree of 911, but 912 is not less than 911.

10. **Given any binary tree with 128 nodes where each node has a *left* pointer and a *right* pointer, how many of these pointers are set to nullptr?** 1 / 1 point

129

✓ **Correct**

The number of node pointers to **nullptr** is equal to the number of nodes plus one. For a tree with just one node (the root), the number of pointers to **nullptr** is two. Adding a child node to any tree replaces one pointer to **nullptr** with a pointer to the new node, which then has two pointers to **nullptr**. Hence, for every new node, there is a net increase of one pointer to **nullptr**.

Congratulations! You passed!
TO PASS 80% or higher

Keep Learning

GRADE
100%

Week 3 Quiz

LATEST SUBMISSION GRADE
100%

1. **Create a binary search tree by inserting the following five values one at a time:** 1 / 1 point

   4 6 5 7 8

   What is the height of this tree?

   (Recall how we calculate the height of a tree or subtree: The height of a leaf node by itself is 0. The height of a non-existent tree is -1. Otherwise, the height of a tree is the longest path length from the root of that tree to any one of its leaves.)

   3

   Correct
   The tree **4 6 5 7 8** has a height of three (e.g. the lineage **4 6 7 8** has three edges between its top node **4** and its bottom node **8**).

2. **Create a binary search tree by inserting the following eight values one at a time:** 1 / 1 point

   3 1 2 4 6 5 7 8

   What is the balance factor of the root node of this tree? (For this question, do not perform any rotations on this tree as you insert the items. It's just a binary search tree, not necessarily a balanced BST such as an AVL tree.)

   2

   Correct
   The root node is **3**. Its left subtree consists of **1 2** and so has a height of one. The root's right subtree consists of **4 6 5 7 8**, where the longest path is **4 6 7 8**, which has a height of three. The balance factor of the root is the height of its right subtree (3) minus the height of the left subtree (1), in this case resulting in a balance factor of 2.

3. **For the binary search tree created by inserting these items in this order: 4 3 5 1 2, which node among 1 through 5 is the deepest node with a balance factor of magnitude two or greater? (For this question, do not perform any balancing rotations as you insert these items.)** 1 / 1 point

   3

   Correct
   Looking at the subtree rooted at node **3**, there is no right child, and so the height of its non-existent right subtree is -1. (Leaf nodes are subtrees of height 0, so non-existent subtrees have height -1 for consistency). The left subtree consists of node **1** and its right child **2**. That is a subtree of height 1.

   Therefore, the balance factor of node 3 is (height of right subtree) - (height of left subtree) = (-1) - (1) = -2.

   Nodes 2 and 5 are leaf nodes, so their balance factors are (-1) - (-1) = 0. Node 1 has a balance factor of (0) - (-1) = 1.

   The root node 4 has a balance factor of (0) - (2) = -2, which is the same balance factor as node 3, but node 3 is deeper in the tree, so node 3 is the correct answer: It is the deepest node whose height balance factor has magnitude of two or greater.

4. **Consider the binary search tree that you constructed in the previous question. If we interpret it now as an AVL tree, it has an imbalance that can be fixed with a rotation. (Remember that we focus on the deepest point of imbalance, where the magnitude of the balance factor is 2 or greater, to perform the rotation.)** 1 / 1 point

   After performing the correct balancing rotation about the node that we identified in the previous question, the resulting tree is identical to which one of the following binary search trees? (We'll describe these other trees by listing the order in which you would insert items to create the trees directly.)

   ○ Inserting **2 1 4 3 5** one node at a time.

- (●) Inserting **4 2 5 1 3** one node at a time.
- ( ) Inserting **3 5 2 4 1** one node at a time. FOO
- ( ) Inserting **3 2 4 1 5** one node at a time.

✓ **Correct**
This tree is indeed better balanced than before, and it's the correct result of performing a left-right rotation about the node 3 in the previous tree. In this new tree, the balance factor of greatest magnitude anywhere is now found at the root node, which has balance factor (0) - (1) = -1.

5. **The code that ensures the balance of an AVL tree after node insertion or removal only checks if the height balance factor is +2 or -2. What happens if the height balance factor of a node in an AVL tree after node insertion or removal is greater that +2 or less than -2?** — 1 / 1 point

- (●) An AVL tree never has a node with a height balance factor greater than +2 or less than -2, even after a node insertion or removal.
- ( ) We ignore nodes in an AVL tree with height balance factor greater than +2 or less than -2 because they are statistically rare and are unstable, such that they are removed as soon as any tree balancing rotation occurs.
- ( ) There is additional code not shown that handles the cases when the height balance factor is greater than +2 or less than -2.
- ( ) When insertion and removal create a node whose height balance factor is greater than +2 or less than -2, that node always has a descendant with a height balance factor equal to +2 or -2 and when all of its descendant nodes are resolved, then its height balance factor will be no greater than +2 or no less than -2.

✓ **Correct**
Every node in an AVL tree has a height balance factor of -1, 0 or 1. Inserting a node can increase the height of a subtree by only 1, and so can change the height balance factor of any node to no more than 2 or no less than -2. Similarly, a node in a any binary search tree is removed by either by deleting a leaf node, shortening a chain of nodes by removing a node with a single child, or replacing a node with its immediate ordered predecessor, and all three of these operations change the height of a subtree by no greater than +1 or no less than -1.

6. **If, after inserting a new node into an AVL tree, you now have a node with a height balance factor of -2 with a child with a height balance factor of -1, which of the following operations should be performed?** — 1 / 1 point

- (●) Right Rotation
- ( ) Right-Left Rotation
- ( ) Left-Right Rotation
- ( ) Left Rotation

✓ **Correct**
Since the parent and child height balance share the same sign, that means that there is a "stick" that needs to be rotated to form a "mountain." Since the shared sign of the height balance factors is negative, this means the left subtree is the cause of the imbalance and should be resolved by a *right* rotation.

7. **If, after inserting a new node into an AVL tree, you now have a node with a height balance factor of -2 with a child with a height balance factor of +1, which of the following operations should be performed?** — 1 / 1 point

- (●) Left-Right Rotation
- ( ) Right Rotation
- ( ) Left Rotation
- ( ) Right-Left Rotation

✓ **Correct**
Since the height balance factors of the parent and child have different signs, this is an "elbow." Since the child's height balance factor is +1, we first perform a left rotation on it to turn it into a stick. Then since the parent's height balance factor is -2, we raise the middle of this "stick" to create a "mountain" by performing a right rotation on it.

8. **Which one of the following is NOT a valid reason to choose the B-Tree representation over a standard AVL binary search tree?** — 1 / 1 point

- ( ) B-Trees require fewer block read accesses for tree operations.
- ( ) B-Trees work faster in networked cloud environments than do AVL trees.
- (●) B-Trees have better algorithmic "Big-O" run-time complexity for the find operation.
- ( ) B-Trees run faster on large data sets than do AVL trees.

✓ **Correct**

While the B-Tree find operation runs in O(log_m n) time, the m ends up being a constant factor and O(log_m n) = O(lg n) as a consequence of the big-O characterization of how run time increases as the number of data items (n) increases.

9. Which of the following statements is NOT true for a B-Tree of order m? 1 / 1 point

- ○ Each node can have at most one more child than key.
- ○ All leaf nodes are at the same level of the B-Tree.
- ◉ Each node can hold an ordered list of as many as m keys.
- ○ Any node that is not the root or a leaf holds at least half of the total number of keys allowed in a node.

✓ **Correct**

This statement is indeed false. In an order-m B-tree, each tree node indeed holds multiple keys, but the number of keys is limited to m-1.

10. If a B-Tree is completely filled, meaning every node holds its maximum number of keys and all non-leaf nodes has the maximum number of children, then what happens when an additional key is inserted into the B-Tree? 1 / 1 point

- ○ A new node containing the new key is added above the previous root and becomes the new root. The new root will have one pointer leading to the old root node.
- ○ A new leaf node is simply added to the B-Tree.
- ○ Every leaf node in the entire B-Tree becomes parent to a new leaf node, but all but one of these leaf nodes are "blank" placeholder nodes that contain zero key values.
- ◉ After searching for the leaf node where the new key should go, the leaf is split in half as two separate leaf nodes, and then the middle value is thrown up to the layer above as an inserted key, and this insertion and rebalancing repeats until a new root key rises to the top, which adds a layer to the tree.

✓ **Correct**

Indeed this must happen to maintain the properties of the B-tree.

Congratulations! You passed!
TO PASS 80% or higher

Keep Learning

GRADE
100%

Week 4 Quiz

LATEST SUBMISSION GRADE
100%

1. **Assume you are storing a complete binary tree as a contiguous list of keys in an array such that the root's key is stored at location 1 of the array, the keys from all of the nodes at the next level of the tree are stored in left-to-right order in subsequent locations in the array, then similarly for all of the nodes of each subsequent level.**

   At what array location would the key stored in the node that is the left child of the right child of the root?

   1 / 1 point

   6

   Correct
   Correct. The root is at position 1. Its right child is at 2*(1) + 1 = 3 and position 3's left child is at 2*(3) = 6.

2. **When using an array to store a complete tree, why is the root node stored at index 1 instead of at the front of the array at index 0?**

   1 / 1 point

   - [x] This makes the math for finding children and parents simpler to compute and to explain.
   - [ ] Array index 0 is used to store the number of nodes in the complete tree stored in the array.
   - [ ] We avoid using index 0 to avoid confusion with the value of 0 (**nullptr**) that we normally store in the child pointer of a node to indicate that child does not exist.
   - [ ] We use index zero as a guard to prevent overstepping the root when propagating up the tree from its leaf nodes, which would cause a memory access fault.

   Correct
   Yep. It is worth wasting one memory location to make programming and documentation simpler and less bug-prone. In particular, this lets us find the parent of a given node simply by using integer division by 2.

3. **When is a binary tree a min-heap?**

   1 / 1 point

   - [ ] When every node's value lies between the maximum value of its left child's subtree and the minimum value of its right child's subtree.
   - [ ] When the leaf nodes represent the smallest values in the tree, and every leaf node is smaller than the root.
   - [x] When every node's value is greater than its parent's value.
   - [ ] When every node's value is less than its parent's value.

   Correct
   A non-empty binary tree is a min-heap if the root is less than either or both of its children (if any), and the subtrees of its children are min-heaps. This is equivalent to the definition from the video lesson and equivalent to the answer.

4. **How should one insert a new value into a heap to most efficiently maintain a balanced tree?**

   1 / 1 point

   - [x] Maintain the heap as a complete tree and insert a new value at the one new node position that keeps the tree as a complete tree. Then continually exchange the new value with the value of its parent until the new value is in node where it is greater than the value of its parent.
   - [ ] Maintain the heap as a balanced binary search tree. Walk down the tree from the root exploring the left children first, then the right children, until a node is found that is greater than the new value. Insert a new node with the new value at that position and make the previous node the left child of that new node. Rebalance the tree if the height balance factor magnitude of the new node or its parent exceeds one.
   - [ ] Maintain the heap as an array. Walk down the tree from the root at position 1 in the array, exploring the left children first, then the right children, until a node position is found whose value is greater than the new value. Copy the value at that position and all subsequent positions in the array to one greater position in the array, and store the new value at that position.
   - [ ] Maintain the heap as an AVL tree. Walk down the tree from the root exploring the left children first, then the right children, until a node is found that is greater than the new value. Insert a new node with the new value at that position and make the previous node the left child of that new node. Then

call the appropriate rotation routine to rebalance the tree if the height balance factor magnitude of the new node or its parent reaches two.

✓ **Correct**

This is the best way to maintain a balanced heap, but requires back propagation to ensure the min-heap properly remains valid.

5. **The removeMin operation removes the root of a min-heap tree. Which of the following implements removeMin efficiently while maintaining a balanced min-heap tree.** 1 / 1 point

- (●) Replace the root value with the value of the last leaf (rightmost node at the bottom level) of a complete binary tree, and delete the last leaf. Then repeatedly exchange this last-leaf value with the smaller of the values of its node's children until this last-leaf value is smaller than the values of its node's children, if any.
- ( ) Set the root value to +infinity. If the left child is smaller than the right child, perform a Right-Rotation, otherwise perform a Left-Rotation. Repeat this process at the new infinity-node location until the infinity node is a leaf, then remove and delete it.
- ( ) Delete the root and if the root has two children, then merge its left subtree with its right subtree by inserting each right subtree node value into the left subtree. Then delete the right subtree.
- ( ) Increment the address used to indicate the base location of the array storing the complete binary tree.

✓ **Correct**

This last-leaf node corresponds to the end of the array so that there are never any missing values in the middle of the array.

6. **How many nodes of a complete binary tree are leaf nodes?** 1 / 1 point

- (●) About half.
- ( ) About a fourth.
- ( ) Unknown. Could be one. Could be all.
- ( ) About the square root.

✓ **Correct**

Let n be the number of nodes in the complete binary tree. If the complete binary tree is also a perfect tree, then all of the leaf nodes are at the bottom level, and there are exactly n/2 + 1/2 of them. (n is odd for a perfect binary tree.)

Now consider deleting these nodes one at a time in an order that keeps the tree complete. The first node you delete is the right child of a parent so it decreases n by one and decreases the number of leaf cells by one. The second node you delete will be that parent's left child, turning the parent into a new leaf node, so decreasing n by one but not decreasing the number of leaf nodes. As you continue removing leaf nodes from right to left on that bottom level, you are reducing n by one but the number of leaf nodes by an average of 1/2 (by one for every right child and by zero for every left child).

Hence about half of a perfect binary tree's nodes are leaf nodes, and this continues as you delete any or all of the nodes in the bottom level in a right-to-left order that keeps the binary tree complete.

7. **Recall that the heapifyDown procedure takes a node index whose children (if any) are heaps, but the value of the node might not satisfy the heap property compared to its children's values. This procedure then swaps the node's value with the smallest child value larger than it (if any), and then calls itself on that smallest child node it just swapped values with to further propagate that value down the heap until it finds a valid location for it.** 1 / 1 point

```
1  template <class T>
2  void Heap<T>::_heapifyDown(int index) {
3      if (!_isLeaf(index)) {
4          T minChildIndex = _minChild(index);
5          if (item_[index] > item_[minChildIndex] ) {
6              std::swap( imem_[index], item_[minChildI]);
7              _heapifyDown(minChildIndex);
8          }
9      }
10 }
```

When you call heapifyDown on a given node, what is the maximum number of times heapifyDown is called (including that first call) to find a valid location for the initial value of that node?

- ( ) The maximum number of times heapifyDown is called is the number of non-leaf nodes in its subtree.
- (●) The maximum number of times heapifyDown is called is one plus the height of the node.
- ( ) heapifyDown is only called once since its children are already heaps.
- ( ) The maximum number of times heapifyDown is called is the number of nodes in its subtree.

✓ **Correct**

heapifyDown is recursive, but it is only called on one of its children, so it walks down only one chain of descendants, not all of its descendants.

chain of descendants, not all of its descendants.

8. **What is the run-time algorithmic complexity of calling heapifyDown on every non-leaf node in a complete tree of n nodes?** 1 / 1 point

- O(1)
- O(n^2)
- O(n lg n)
- O(n) ✓

Correct

The run-time of calling heapifyDown on a node is proportional to the height of the node. About half of the nodes are leaf nodes, about a quarter have height 1, about an eighth have height 2, about a sixteenth have height 3, and so on. This summation of heights converges to n, the number of nodes in the tree. Hence running heapifyDown on every non-leaf node has a run-time complexity of O(n).

9. **Which of the following is the fastest way to build a heap of n items?** 1 / 1 point

- Create a complete tree of the items in any order, then call heapifyDown on every non-leaf node from the bottom of the tree up to the root. ✓
- Create a complete tree of the items in any order, then call heapifyUp on every node in the tree from the root down to the leaf nodes.
- Create a heap with a single node holding the first item. Then insert the remaining n-1 items into the heap.
- Create a complete tree of the items in any order. If this is not a heap, then swap the items between two randomly chosen nodes and check again. Repeat the random swapping until you get a heap.

Correct

As we showed before, this runs in O(n) time.

10. **Which of the following is NOT a step of the heap sort algorithm?** 1 / 1 point

- Remove the root node.
- Load the data in any order into a complete tree.
- Run heapifyDown on every non-leaf node.
- Insert the next item into the current heap. ✓

Correct

The heap sort algorithm, as presented in the video lessons, does not rely on the insertion of any item into an existing heap.

Congratulations! You passed!
TO PASS 80% or higher

Keep Learning

GRADE
90%

Week 1 Quiz

LATEST SUBMISSION GRADE
90%

1. **According to video lesson 1.1.1, a hash table consists of three things. Which of these was NOT one of those three things?** — 1 / 1 point

- (●) Encryption
- ( ) Collision handling
- ( ) A hash function
- ( ) An array

Correct
That's correct: Encryption is not involved in our discussion of hash tables. While hash functions are involved in encryption techniques, encryption and hashing are not the same thing, and encryption is not a requirement of a hash table.

A hash table implementation uses a hash function to translate a key into an index. The index would be used with an underlying array for data storage, which allows constant-time access based on the index. Collision handling ensures that when different keys result in the same hash value, they are translated to different indices.

2. **Given a hash function h(key) that produces an index into an array of size N, and given two different key values key1 and key2, the Simple Uniform Hashing Assumption states which of the following?** — 1 / 1 point

- ( ) If h(key1) == h(key2) then h needs a running time of O(N) to complete.
- (●) The probability that h(key1) == h(key2) is 1/N.
- ( ) If h(key1) == h(key2) then h needs a running time of O(lg N) to complete.
- ( ) The probability that h(key1) == h(key2) is 0.

Correct
The SUHA states that P(h(key1) == h(key2)) = 1/N.

3. **According to video lesson 1.1.2, which of the following is a *good* hash function h(key) that translates any 32-bit unsigned integer key into an index into an 8 element array?** — 1 / 1 point

(Note that an expression like "2 & 3" uses the bitwise-AND operator, which gives the result of comparing every bit in the two operands using the concept of "AND" from Boolean logic; for example, in Boolean logic with binary numbers, 10 AND 11 gives 10: for the first digit, 1 AND 1 yields 1, while for the second digit, 0 AND 1 yields 0. An expression like "4 % 8" uses the remainder operator that give the remainder from integer division; for example, 4 % 8 yields 4, which is the remainder of 4/8. In some cases, these two different operators give similar results. Think about why that is.)

( )
```
int h(uint key) { return rand() % 8; }
```

( )
```
int h(uint key) { return max(key,7); }
```

(●)
```
int h(uint key) { return key & 7; }
```

( )
```
int h(uint key) {
    int index = 5;
    while (key--)
        index = (index + 5) % 8
    return index;
}
```

Correct
This always generates the same output given the same input, and it has a uniform chance of collision. It also runs in constant time relative to the length of the input integer (that is, relative to the number of bits, without respect to the magnitude of the integer).

Note that in binary, the number 7 is 0000...0111. (The leading digits are all zero, followed by three 1 digits, because these place values represent 4+2+1.) When you do "key & 7", the result

three 1 digits, because these place values represent 4+2+1.) When you do "key & 7", the result will have leading zeros, and the rightmost three digits will be the same as those of key. Because this results in values between 0 and 7, it's similar to taking the remainder of division by 8. That is, "key & 7" should give the same result as "key % 8".

Bitwise operations like this can be somewhat faster than arithmetic operations, but you have to be careful about the specific data types and the type of computing platform you are compiling for. Note that this trick only works for some right-hand values as well, based on how they are represented in binary. These tricks are not always portable from one system architecture to another.

4. **Suppose you have a good hash function h(key) that returns an index into an array of size N. If you store values in a linked list in the array to manage collisions, and you have already stored n values, then what is the expected run time to store a new value into the hash table?** 1 / 1 point

- ○ O(n)
- ○ O(n/N)
- ○ O(N)
- ◉ O(1)

✓ **Correct**
Storing a new value takes constant time because the hash function runs in constant time and inserting a new value at the head of a linked list takes constant time.

5. **Suppose you have a good hash function h(key) that returns an index into an array of size N. If you store values in a linked list in the array to manage collisions, and you have already stored n values, then what is the expected run time to find the value in the hash table corresponding to a given key?** 1 / 1 point

- ◉ O(n/N)
- ○ O(N)
- ○ O(n)
- ○ O(1)

✓ **Correct**
This is the "load factor" of the hash table, and is the average length of the linked lists stored at each array element. Since the lists are unordered, It would take O(n/N) time to look at all of the elements of the list to see if the desired (key/value) pair is in the list.

6. **Which one of the following four hashing operations would run faster than the others?** 1 / 1 point

- ◉ Finding a value in a hash table of 100 values stored in an array of 1,000 elements.
- ○ Finding a value in a hash table of 2 values stored in an array of 2 elements.
- ○ Finding a value in a hash table of 20 values stored in an array of 100 elements.
- ○ Finding a value in a hash table of 4 values stored in an array of 8 elements.

✓ **Correct**
The load factor is 100/1,000 = 0.1 which is less than the other options.

7. **When storing a new value in a hash table, linear probing handles collisions by finding the next unfilled array element. Which of the following is the main drawback of linear probing?** 1 / 1 point

- ◉ Even using a good hash function, contiguous portions of the array will become filled, causing a lot of additional probing in search of the next available unused element in the array.
- ○ The array only stores values, so when retrieving the value corresponding to a key, there is no way to know if the value at h(key) is the proper value, or if it is one of the values at a subsequent array location.
- ○ There may not be an available slot in the array.
- ○ If the hash function returns an index near the end of the array, there might not be an available slot before the end of the array is reached.

✓ **Correct**
This happens because the hashing distributes values uniformly in the array, but the linear probing fills in gaps between the locations of previous values, which makes the situation worse for later values added to the array.

8. **When using double hashing to store a value in a hash table, if the hash function returns an array location that already stores a previous value, then a new array location is found as the hash function of the current array location. Why?** 0 / 1 point

- ○ Double hashing reduces the clumping that can occur with linear probing.
- ○ Since the hash function runs in constant time, double hashing runs in O(1) time.

- (selected) Double hashing reduces the chance of a hash function collision on subsequent additions to the hash table.
- Only one additional hash function is called to find an available slot in the array whereas linear probing requires an unknown number of array checks to find an available slot.

Incorrect
Double hashing does not affect the chance of a hash function collision. Because of SUFA, any addition to the hash table has the same chance of causing a subsequent collision of the hash function.

9. **Which of the following data structures would be the better choice to implement a memory cache, where a block of global memory (indicated by higher order bits of the memory address) are mapped to the location of a block of faster local memory.** — 1 / 1 point

- A hash table implemented with separate chaining, using an array of linked lists.
- (selected) A hash table implemented with double hashing.
- A hash table implemented with linear probing.
- An AVL tree.

Correct
Double hashing would be a good strategy because the cache addresses are quite small and compactly stored in the array. Furthermore, double hashing is more efficient than linear probing, which suffers from clumping.

10. **Which of the following data structures would be the better choice to implement a dictionary that not only returns the definition of a word but also returns the next word following that word (in lexical order) in the dictionary.** — 1 / 1 point

- (selected) An AVL tree.
- A hash table implemented with separate chaining, using an array of linked lists.
- A hash table implemented with linear probing.
- A hash table implemented with double hashing.

Correct
While the AVL tree needs O(log n) time to find the definition of the word, which is worse than the performance of a hash table, the AVL tree can find the next word in lexical order in O(log n) time whereas any hash table would need O(N) steps to find the next word in lexical order.

Congratulations! You passed!
TO PASS 80% or higher

Keep Learning

GRADE
100%

Week 2 Quiz

LATEST SUBMISSION GRADE
100%

1. **Using the convention followed by the video lessons, given three disjoint sets (1,3,5,7), (2,8) and (4,6), which one of these sets would be referenced by the value 3?** — 1 / 1 point

- (1,3,5,7)
- (2,8)
- (4,6)
- None of the above.

Correct
A disjoint set is referenced by one of the values in the set.

2. **What is the union of the disjoint sets (1,3,5,7) and (2,8)?** — 1 / 1 point

- (3,11)
- (1,2,3,5,7,8)
- (2,6,8,10,14,16,24,40,56)
- ((1,2),(1,8),(3,2),(3,8),(5,2),(5,8),(7,2),(7,8))

Correct
The union of disjoint sets contains only each element from the first set and each element from the second set.

3. **What happens when you take the union of two disjoint sets that contain the same value?** — 1 / 1 point

- Any element found in both disjoint sets will appear twice in the union of these two disjoint sets.
- The elements cancel and neither appears in the union of the two disjoint sets.
- The union operation must first check to see if the same element appears in both disjoint sets and then ensures the element appears only once in the resulting union set.
- Two different disjoint sets by definition can never share the same value.

Correct
Disjoint sets represent a partitioning of unique values into subsets that do not have any items in common. That is, each value belongs to exactly one of the sets. This is why each element can be used as an array index look up its "up-tree" parent, which represents the set the element belongs to.

4. **According to the distjoint set array representation in the video lessons, Which of the following arrays would NOT be a valid representation of the disjoint set (1,3,5,7)?** — 1 / 1 point

-1	**-1**	**1**	**-1**	**1**	**-1**	**1**	**-1**
1	2	3	4	5	6	7	8

5	**-1**	**-1**	**-1**	**3**	**-1**	**1**	**-1**
1	2	3	4	5	6	7	8

-1	**-1**	**1**	**-1**	**3**	**-1**	**5**	**-1**
1	2	3	4	5	6	7	8

3	**-1**	**5**	**-1**	**7**	**-1**	**1**	**-1**
1	2	3	4	5	6	7	8

Correct
This is indeed not valid because there is no root of the up-tree. Element 1 points to element 3 which points to element 5 which points to element 7 which points to element 1, so no element in this disjoint set is the root and would represent the disjoint set.

5. **When encoding height into the root of an up-tree, what value should be placed in element 7 of the following array?** 1 / 1 point

3	-1	7	-1	7	-1	???
1	2	3	4	5	6	7

- (•) -3
- ( ) -2
- ( ) -1
- ( ) -4

✓ **Correct**
The value should be equal to -1 minus the height. A singleton disjoint set would have height zero but there is no -0 and 0 would point to the 0th element of the array, so we increment the height by one and negate it before storing it in the root of the up-tree.

6. **When encoding size into the root of an up-tree, what value should be placed in element 7 of the following array?** 1 / 1 point

3	-1	7	-1	7	-1	???
1	2	3	4	5	6	7

- (•) -4
- ( ) -1
- ( ) -3
- ( ) -2

✓ **Correct**
Correct. This up-tree represents the disjoint set (1,3,5,7) which has four elements.

7. **When computing the union of two disjoint sets represented as up-trees in an array, (using proper path compression) which of these strategies results in a better overall run time complexity than the other options?** 1 / 1 point

- ( ) Always make the up-tree with fewer elements a subtree of the root of the up-tree with more elements.
- ( ) Always make the up-tree with a shorter height a subtree of the root of the up-tree with a larger height.
- ( ) The overall run time complexity is not affected by which up-tree is chosen to become a subtree of the other up-tree.
- (•) Using either size or height strategies above results in the same overall run time complexity.

✓ **Correct**
Indeed, the overall time complexity is the same when using either size or height strategies.

8. **Recall that the iterated log function is denoted log*(n) and is defined to be** 1 / 1 point

- **0 for n <= 1, and**
- **1 + log*(log(n)) for n > 1.**

Let lg*(n) be this iterated log function computed using base 2 logarithms.

Blue Waters, housed at the University of Illinois, is the fastest university supercomputer in the world. It can run about 2^53 (about 13 quadrillion) instructions in a second. There are about 2^11 seconds in half an hour, so Blue Waters would run 2^64 instructions in about half an hour.

Which one of the following is equal to lg*(2^64)?

- ( ) 6
- ( ) 64
- (•) 5
- ( ) 65536

✓ **Correct**
lg*(2^64) = 1 + lg*(64) = 1 + 1 + lg*(6) = 1+ 1 + 1 + lg*(~2.6) = 1 + 1 + 1 + 1 + lg*(1.4) = 1 + 1 + 1 + 1 + 1 + lg*(0.5) = 5

9. **Which of these is considered the least run-time complexity?** 1 / 1 point

- O(log* N)
- O(1) (selected)
- O(log N)
- O(log log N)

✓ **Correct**

Constant time is indeed better than iterated-log time complexity, which is "practically" constant time, but does grow ever so slightly as N increases.

10. **Which of the following best describes "path compression" as described in the video lessons to accelerate disjoint set operations? (Here we say "parent pointer" to mean whatever form of indirection is used to refer from a child to its parent; this could be a literal pointer or it could be an array index as in the lectures.)** 1 / 1 point

- When the root of the up-tree containing an element is found, the element and all of its siblings that share the same parent have their parent pointers reset to point to the root node.
- When traversing the up-tree from an element to its root, if any elements in the traversal (including the first element, but excluding the root itself) do not point directly to the root as their parent yet, they will have their parent pointer changed to point directly to the root. (selected)
- When the root of an element's node is found, all of the descendants of the root have their parent pointer set to the root.
- When the root of the up-tree containing an element is found, both the element and its parent will always have their parent pointers set to point to the root node.

✓ **Correct**

That's right: Path compression only flattens the lineage of nodes in an up-tree from an element to the root, and not all of the elements in the up-tree every time. This has amortized benefits as the data structure is optimized over the process of several union and find operations.

Congratulations! You passed!
TO PASS 80% or higher

Keep Learning

GRADE
80%

Week 3 Quiz

LATEST SUBMISSION GRADE
80%

1. **Let G = (V,E) be a simple graph consisting of a set of vertices V and a set of (undirected) edges E where each edge is a set of two vertices. Which one of the following is not a simple graph?** — 1 / 1 point

- G = (V = (a,b,c), E = ((a,b),(b,c),(a,c)))
- G = (V = (a,b,c), E = ((a,b), (a,c), (b,a), (b,c), (a,c), (b,c)))
- G = (V = (a,b,c), E = ())
- G = (V = (a,b,c), E = ((a,b)))

Correct
This is not a *simple* graph because the same edge between a and b appears twice, once as (a,b) and a second time as (b,a). Since these are sets, (a,b) == (b,a).

2. **For a simple graph with n vertices, what is the worst case (largest possible) for the number of edges?** — 1 / 1 point

- O(n^2)
- O(2^n)
- O(n log n)
- O(n)

Correct
Recall that the adjacency matrix has one entry per edge in its upper triangular portion. There are n^2 elements in the n x n adjacency matrix, and about 1/2 n^2 elements in its upper triangular portion, and O(1/2 n^2) == O(n^2).

3. **Which graph representation has a better worst-case storage complexity than the others for storing a simple graph of n vertices?** — 0 / 1 point

- Edge List
- Adjacency Matrix
- Adjacency List
- All three graph representations have the same worst-space storage complexity for a simple graph of n nodes.

Incorrect
The adjacency matrix requires O(n^2) space to store at least the upper-triangular portion of the n x n matrix. This is not better than at least one of the other options.

4. **Suppose you have a rapid data feed that requires you to add new data point vertices quickly to a graph representation. Which graph representation would you NOT want to utilize?** — 1 / 1 point

- Edge List
- Adjacency Matrix
- Adjacency List
- All three graph representations have the same time complexity for adding vertices to a simple graph.

Correct
The adjacency matrix requires linear time, O(n), to add a vertex because the addition requires new entries to be placed in a new row and a new column of the matrix, and there are n elements in the new row and n elements in the new column. This means that as the number of vertices grows in the graph, it will take longer to add a new vertex, which is not a very good choice when processing a data feed.

5. **Suppose you have a rapid data feed that requires you to remove existing data point vertices (and any of their edges to other vertices) quickly to a graph representation. Which graph representation would you WANT to utilize?** — 1 / 1 point

- ○ Edge List
- ○ Adjacency Matrix
- ◉ Adjacency List
- ○ All three representations have the same time complexity for removing a vertex from a simple graph of n vertices.

✓ **Correct**

Since the adjacency list has a list of the edges the removed vertex shares with other vertices, it only needs time proportional to the degree of the removed vertex. In the worst case, that vertex could be connected to all of the other vertices and so require O(n) time, but in the typical case the degree will be less and the adjacency list is a better choice than the adjacency matrix.

6. **Suppose you want to implement a function called neighbors(v) that returns the list of vertices that share an edge with vertex v. Which representation would be the better choice for implementing this neighbors() function?** — 1 / 1 point

- ○ Edge List
- ○ Adjacency Matrix
- ◉ Adjacency List
- ○ All three representations result in the same time complexity for the neighbor() function.

✓ **Correct**

The adjacency list requires a simple walk through the list of pointers to adjacent edges to find the neighboring vertices. This representation has an "output sensitive" running time meaning it runs as fast as possible based on the minimum amount of time needed to output the result.

7. **Suppose you want to implement a function called neighborsQ(v1,v2) that returns true only if vertices v1 and v2 share an edge. Which representation would be the better choice for implementing this neighborsQ() function?** — 1 / 1 point

- ○ Edge List
- ◉ Adjacency Matrix
- ○ Adjacency List
- ○ All three representations support the same time complexity for implementing the neighborQ() function.

✓ **Correct**

The neighborsQ(v1,v2) function can simply lookup the appropriate v1,v2 entry in the adjacency matrix, which takes constant O(1) time. This representation supports the fastest method for implementing this query.

8. **Which of these edge lists has a vertex of the highest degree?** — 1 / 1 point

- ○ (d,b), (g,a), (h,f), (c, e)
- ◉ (a,b), (b, c), (d, b), (g, b)
- ○ (a, c), (e, g), (c, e), (g, a)
- ○ (a, b), (a, c), (a, d), (b, d)

✓ **Correct**

Vertex b has degree four.

9. **Which adjacency matrix corresponds to the edge list: (1,2), (2,3), (3,4), (4,1) (where the rows/columns of the adjacency matrix follow the same order as the vertex indices)?** — 1 / 1 point

○

1	0	0	1
	1	0	0
		1	0
			1

○

0	1	1	0
	0	1	1
		0	1
			0

○

0	0	1	1
	0	1	1

		0	0
			0

0	1	0	1
	0	1	0
		0	1
			0

Correct

10. **Which graph representation would be the best choice for implementing a procedure that only needs to build a graph from a stream of events.** 0 / 1 point

- Edge List
- Adjacency Matrix
- Adjacency List
- All three representations would share the same storage and time complexity for the procedure.

Incorrect

The adjacency list has the same storage complexity as the edge list (and better in general than the adjacency matrix). The adjacency list also has the same time complexity as the edge list (and better than the adjacency matrix) for the needed operations. However, the adjacency list adds additional storage and code over the edge list with no improvement in time complexity, so it is not the better option for implementing the procedure.

Congratulations! You passed!
TO PASS 80% or higher

Keep Learning

GRADE
90%

Week 4 Quiz

LATEST SUBMISSION GRADE
90%

1. **Which of these algorithms can be used to count the number of connected components in a graph?** — 1 / 1 point

- ○ Count the number of times a breadth first traversal is started on every vertex of a graph that has not been visited by a previous breadth first traversal.
- ○ Count the number of times a depth first traversal is started on every vertex of a graph that has not been visited by a previous breadth first traversal.
- ◉ All of the above
- ○ None of the above

✓ **Correct**
Both breadth first and depth first searches visit every vertex connected to a starting vertex by a path of edges, and thus visit every vertex in a connected component of the graph.

2. **Which elements encountered by a breadth first search can be used to detect a cycle in the graph?** — 1 / 1 point

- ○ Discovered edges that were previously unexplored by the traversal have been added to the breadth-first traversal.
- ○ Unexplored vertices that have been encountered by the traversal of a previously unexplored edge.
- ○ Unexplored edges to unexplored vertices that remain so after completion of the breadth first search.
- ◉ Previously visited vertices that have been encountered again via a previously unexplored edge.

✓ **Correct**
A breadth first traversal returns a spanning tree of each connected component of the graph. Any edge that is not part of the breadth first search (e.g. not marked discovered) will connect one portion of the tree to another forming a cycle. Thus all unexplored edges, including ones ignored because they reach a previously visited vertex will create a cycle if added to the breadth first search.

3. **A breadth first traversal starting at vertex v1 of a graph can be used to find which ones of the following?** — 1 / 1 point

- ◉ The shortest path (in terms of # of edges) between vertex v1 and any other vertex in the graph.
- ○ The shortest path (in terms of # of edges) between any two vertices in the graph.
- ○ All of the above.
- ○ None of the above.

✓ **Correct**
A breadth first search adds vertices connected to the current frontier of vertices using a queue, so that all vertices a given number of edges away from the start vertex must have been processed through the queue before the next wave of vertices can be processed. Since vertices are added in this edge-length order, the breadth first traversal finds the shortest path from the start vertex to any vertex, but this is only true for the start vertex.

4. **Which traversal method has a better run time complexity to visit every vertex in a graph?** — 1 / 1 point

- ○ Breadth First Traversal
- ○ Depth First Traversal
- ◉ Both have the same run time complexity.
- ○ Neither traversal method will necessarily visit every vertex in a graph.

✓ **Correct**
Both options run in time O(n + m) for n vertices and m edges.

5. **The breadth first traversal of a connected graph returns a spanning tree for that graph that contains every vertex. If the graph has weighted edges, which of the following modifications is the simplest that produces a minimum spanning tree for the graph of weighted edges.** — 1 / 1 point

- The queue is replaced by a priority queue that keeps track of the total weight encountered by the current traversal plus each of the edges that connects a vertex to the current breadth first traversal.
- An ordinary breadth first traversal is run from each vertex (as its start vertex) and the resulting spanning tree with the least total weight is the minimum spanning tree.
- No modification is necessary because a breadth first traversal always returns a minimum spanning tree.
- (selected) The queue is replaced by a priority queue that keeps track of the least-weight edge that connects a vertex to the current breadth first traversal.

✓ **Correct**
A minimum spanning tree for a weighted graph can be found through a greedy breadth-first algorithm that simply chooses from the entire queue the least weight edge to add.

6. True of false: a connected directed graph with no cycles is a tree. — 0 / 1 point

- (selected) True
- False

! **Incorrect**
A directed graph such as A -> B, A --> C, B --> D and C --> D is connected and has no cycle, but is not a tree because there are multiple paths from vertex A to vertex D.

7. For which one case below will Dijkstra's algorithm fail to produce a shortest path? — 1 / 1 point

- A connected graph where every node is connected to every other node by an edge, and all edges have the same weight.
- A connected graph that is a tree of negative edges.
- A connected graph with an acyclic path of negative weight edges.
- (selected) A connected graph with a cycle of negative weight edges.

✓ **Correct**
Dijkstra's algorithm would continue to cycle through these negative edges, accumulating a shorter path in terms of length as measured by edge weight path sum and would never terminate.

8. Which of the following is a true statement about Dijkstra's algorithm? — 1 / 1 point

- Dijkstra's algorithm finds the shortest unweighted path, if it exists, between a start vertex and any other vertex, but only for an undirected graph.
- Dijkstra's algorithm finds the shortest weighted path, if it exists, between a start vertex and any other vertices, but only for an undirected graph.
- (selected) Dijkstra's algorithm finds the shortest weighted path, if it exists, between a start vertex and any other vertices in a directed graph.
- Dijkstra's algorithm finds the shortest weighted path, if it exists, between a any two vertices in a directed connected graph.

✓ **Correct**
Dijkstra's algorithm indeed works with directed edges, only following an edge along its direction from a start vertex to an end vertex.

9. Which of the following is the optimal run time complexity to find the shortest path, if it exists, from a vertex to all of the other vertices in a weighted, directed graph of n vertices and m edges. — 1 / 1 point

- O(m + lg n)
- (selected) O(m + n lg n)
- O(m + n)
- O(n)

✓ **Correct**
This is the running time for Dijkstra's algorithm which is optimal.

10. Which of the following is the fastest way in a simple graph to find the shortest path from a start vertex to an end vertex along a path constrained to pass through a third landmark vertex along the way? — 1 / 1 point

- Two runs of Dijkstra's algorithm.
- (selected) A single run of a breadth first traversal.

- ○ Three runs of a breadth first traversal.
- ○ A single run of Dijkstra's algorithm.

✓ **Correct**

A single breadth first traversal from the landmark vertex finds the shortest paths from it to the start vertex and the end vertex, and their combination is the shortest path from start to end that also visits the landmark.